

Network Latency Debugging and Optimization

Complete guide to identify, debug, and fix network latency issues in distributed systems.

Understanding Network Latency

Latency = Time for data to travel from source to destination

Components:

- DNS lookup time
- TCP connection establishment
- TLS handshake (if HTTPS)
- Request transmission time
- Server processing time
- Response transmission time
- Network hops between nodes

Step 1: Measure and Identify

1.1 Basic Latency Measurement

Using curl:

```
# Create curl-format.txt
cat > curl-format.txt << EOF
    time_namelookup:  %{time_namelookup}s\n
    time_connect:     %{time_connect}s\n
    time_appconnect:   %{time_appconnect}s\n
    time_pretransfer:  %{time_pretransfer}s\n
    time_redirect:     %{time_redirect}s\n
    time_starttransfer:  %{time_starttransfer}s\n
                        -----\n
    time_total:       %{time_total}s\n
EOF

# Measure latency
curl -w "@curl-format.txt" -o /dev/null -s https://api.example.com/users
```

Output:

```
time_namelookup: 0.012s    ← DNS lookup
time_connect:    0.045s    ← TCP connection
time_appconnect: 0.089s    ← TLS handshake
time_pretransfer: 0.089s   ← Ready to transfer
time_redirect:   0.000s    ← Redirects
```

```
time_starttransfer: 0.523s    ← First byte received (TTFB)
-----
time_total: 0.678s    ← Total time
```

Analysis:

- DNS (12ms) - Normal
- TCP (33ms) - Normal
- TLS (44ms) - Normal
- TTFB (434ms) - **HIGH! Problem is server or network**
- Transfer (155ms) - Normal

1.2 Application-Level Measurement

Python:

```
import time
import requests

def measure_latency(url):
    timings = {}

    # DNS + Connect
    start = time.time()
    response = requests.get(url)
    total = time.time() - start

    timings['total'] = total
    timings['status'] = response.status_code
    timings['size'] = len(response.content)

    return timings

# Test
result = measure_latency('https://api.example.com/users')
print(f"Total: {result['total']:.3f}s")
print(f"Status: {result['status']}")
print(f"Size: {result['size']} bytes")
```

Advanced with breakdown:

```
import time
import socket
import requests
from requests.adapters import HTTPAdapter

def detailed_latency(url):
    timings = {}
```

```

# 1. DNS lookup
dns_start = time.time()
hostname = url.split('://')[1].split('/')[0].split(':')[0]
socket.gethostbyname(hostname)
timings['dns'] = time.time() - dns_start

# 2. Full request
total_start = time.time()

# Track connection time
session = requests.Session()
adapter = HTTPAdapter(pool_connections=1, pool_maxsize=1)
session.mount('http://', adapter)
session.mount('https://', adapter)

response = session.get(url)
timings['total'] = time.time() - total_start

# Calculate server processing time (TTFB - connection)
# Approximate: total - dns - reasonable connection time
timings['server_processing'] = timings['total'] - timings['dns'] - 0.1

return timings

# Test
timings = detailed_latency('https://api.example.com/users')
print(f"DNS: {timings['dns']*1000:.1f}ms")
print(f"Server: {timings['server_processing']*1000:.1f}ms")
print(f"Total: {timings['total']*1000:.1f}ms")

```

1.3 Distributed Tracing

Using OpenTelemetry:

```

from opentelemetry import trace
from opentelemetry.instrumentation.requests import RequestsInstrumentor
from opentelemetry.sdk.trace import TracerProvider
from opentelemetry.sdk.trace.export import ConsoleSpanExporter, BatchSpanProcessor
from opentelemetry.exporter.jaeger.thrift import JaegerExporter

# Setup tracing
trace.set_tracer_provider(TracerProvider())

# Export to Jaeger
jaeger_exporter = JaegerExporter(
    agent_host_name='localhost',
    agent_port=6831,
)
trace.get_tracer_provider().add_span_processor(
    BatchSpanProcessor(jaeger_exporter)
)

```

```
# Auto-instrument requests
RequestsInstrumentor().instrument()

# Your code
tracer = trace.get_tracer(__name__)

with tracer.start_as_current_span("api-call"):
    with tracer.start_as_current_span("fetch-user"):
        user = requests.get('http://user-service/users/123')

    with tracer.start_as_current_span("fetch-orders"):
        orders = requests.get('http://order-service/orders?user=123')

    with tracer.start_as_current_span("process-data"):
        result = process(user, orders)
```

Jaeger UI shows:

```
api-call (650ms)
├── fetch-user (200ms)
│   ├── DNS lookup (10ms)
│   ├── TCP connect (30ms)
│   ├── TLS handshake (40ms)
│   └── Server processing (120ms)
├── fetch-orders (400ms) ← BOTTLENECK
│   ├── DNS lookup (5ms)
│   ├── TCP connect (25ms)
│   ├── TLS handshake (35ms)
│   └── Server processing (335ms) ← Problem here!
└── process-data (50ms)
```

1.4 Network Diagnostic Tools

Ping (ICMP):

```
# Test connectivity and latency
ping -c 10 api.example.com

# Output:
# 10 packets transmitted, 10 received, 0% packet loss
# round-trip min/avg/max/stddev = 45.2/52.8/68.3/6.4 ms
```

Traceroute (Path analysis):

```
# Show route and latency per hop
traceroute api.example.com
```

```
# Output:
# 1  router (192.168.1.1)  1.234 ms
# 2  isp-gateway (10.0.0.1) 12.456 ms
# 3  isp-core (203.0.113.1) 45.678 ms ← High latency here
# 4  cdn-node (198.51.100.1) 48.901 ms
# 5  api.example.com (93.184.216.34) 52.345 ms
```

MTR (Continuous traceroute):

```
# Combined ping + traceroute
mtr -r -c 100 api.example.com

# Shows packet loss and latency per hop
```

Netstat (Connection state):

```
# Check established connections
netstat -an | grep ESTABLISHED

# Check connection queue
ss -s
```

Step 2: Common Causes and Fixes

2.1 DNS Lookup Latency

Problem: DNS resolution taking too long (>100ms)

Identify:

```
# Test DNS lookup time
time nslookup api.example.com

# Or using dig
dig api.example.com +stats
```

Causes:

- Slow DNS server
- No DNS caching
- Too many DNS lookups

Fix 1: Use Faster DNS Server

```
# Switch to Cloudflare DNS (1.1.1.1) or Google (8.8.8.8)
# Linux: /etc/resolv.conf
nameserver 1.1.1.1
nameserver 8.8.8.8

# Test improvement
time nslookup api.example.com
```

Fix 2: DNS Caching

```
# Python: Use connection pooling to reuse connections
import requests
from requests.adapters import HTTPAdapter
from requests.packages.urllib3.util.retry import Retry

session = requests.Session()

# Connection pooling (reuses DNS, TCP, TLS)
adapter = HTTPAdapter(
    pool_connections=10,
    pool_maxsize=100,
    pool_block=False
)
session.mount('http://', adapter)
session.mount('https://', adapter)

# Reuse session for all requests
response = session.get('http://api.example.com/users')
```

Fix 3: Local DNS Cache

```
# Install dnsmasq (Linux)
sudo apt install dnsmasq
sudo systemctl start dnsmasq

# Configure to use local cache
echo "nameserver 127.0.0.1" > /etc/resolv.conf
```

Results:

- DNS lookup: 50ms → 1ms (50x faster)

2.2 TCP Connection Overhead

Problem: TCP handshake taking long (>100ms per connection)

Identify:

```
import time
import socket

start = time.time()
sock = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
sock.connect(('api.example.com', 443))
connect_time = time.time() - start
sock.close()

print(f"TCP connect: {connect_time*1000:.1f}ms")
```

Causes:

- Geographic distance
- Creating new connections for each request
- Network congestion

Fix 1: Keep-Alive / Connection Pooling

```
# Python: Reuse connections
import requests

session = requests.Session()
session.headers.update({'Connection': 'keep-alive'})

# All requests reuse same TCP connection
for i in range(10):
    response = session.get('http://api.example.com/users')
    # No TCP handshake after first request
```

Node.js:

```
const http = require('http');

// Enable keep-alive
const agent = new http.Agent({
  keepAlive: true,
  maxSockets: 50,
  maxFreeSockets: 10,
  timeout: 60000
});

// Reuse connections
http.get({
  hostname: 'api.example.com',
  port: 80,
  path: '/users',
  agent: agent
```

```
}, (res) => {  
    // ...  
});
```

Fix 2: TCP Fast Open (TFO)

```
# Enable TFO on Linux  
sudo sysctl -w net.ipv4.tcp_fastopen=3  
  
# On server and client, reduces 1 RTT from handshake
```

Fix 3: Use HTTP/2 or HTTP/3

```
import httpx  
  
# HTTP/2 multiplexes multiple requests over single connection  
async with httpx.AsyncClient(http2=True) as client:  
    responses = await asyncio.gather(  
        client.get('http://api.example.com/users'),  
        client.get('http://api.example.com/orders'),  
        client.get('http://api.example.com/products')  
    )  
# All 3 requests over same TCP connection
```

Results:

- First request: 100ms (TCP handshake)
- Subsequent requests: 0ms (connection reused)
- HTTP/2: All requests in parallel over 1 connection

2.3 TLS/SSL Handshake Overhead

Problem: TLS handshake taking long (>100ms)

Identify:

```
# Measure TLS handshake  
openssl s_time -connect api.example.com:443  
  
# Or with curl  
curl -w "TLS: %{time_appconnect}s\n" -o /dev/null -s https://api.example.com
```

Causes:

- Full TLS handshake on every connection

- Weak cipher suites
- Certificate validation overhead

Fix 1: TLS Session Resumption

Server-side (Nginx):

```
# nginx.conf
ssl_session_cache shared:SSL:10m;
ssl_session_timeout 10m;
ssl_session_tickets on;
```

Client-side (Python):

```
import requests

# Session reuses TLS session
session = requests.Session()

# First request: full TLS handshake (100ms)
response1 = session.get('https://api.example.com/users')

# Second request: TLS session resumed (10ms)
response2 = session.get('https://api.example.com/orders')
```

Fix 2: HTTP/2 (Persistent Connections)

```
import httpx

# HTTP/2 reuses TLS connection
async with httpx.AsyncClient(http2=True) as client:
    # Only one TLS handshake for all requests
    r1 = await client.get('https://api.example.com/users')
    r2 = await client.get('https://api.example.com/orders')
    r3 = await client.get('https://api.example.com/products')
```

Fix 3: Optimize Cipher Suites

```
# Use faster ciphers
ssl_ciphers 'ECDHE-ECDSA-AES128-GCM-SHA256:ECDHE-RSA-AES128-GCM-SHA256';
ssl_prefer_server_ciphers on;
```

Results:

- First request: 100ms TLS handshake

- Subsequent: 10ms (session resumed)
 - HTTP/2: Single TLS handshake for all requests
-

2.4 Physical Distance / Geographic Latency

Problem: High latency due to physical distance

Identify:

```
# Check round-trip time
ping api.example.com

# Speed of light limit:
# Distance / Speed ≈ minimum latency
# US West to East: ~50ms
# US to Europe: ~80-120ms
# US to Asia: ~150-250ms
```

Fix 1: CDN (Content Delivery Network)

```
// Use CDN for static assets
// CDN serves from nearest edge location

// Before: US server (150ms from Asia)
<script src="https://myapp.com/bundle.js"></script>

// After: CDN edge node (20ms from anywhere)
<script src="https://cdn.myapp.com/bundle.js"></script>
```

Fix 2: Regional Deployments

```
# Deploy services in multiple regions
# Route users to nearest region

# AWS Route53 latency-based routing
Type: A
Name: api.example.com
Routing: Latency
Records:
  - us-west-1: 54.183.255.128 (US users)
  - eu-west-1: 34.253.12.34 (EU users)
  - ap-south-1: 13.126.245.89 (Asia users)
```

Fix 3: Edge Computing

```
// Cloudflare Workers / AWS Lambda@Edge
// Run code at edge locations

export default {
  async fetch(request) {
    // This runs at edge (close to user)
    const response = await fetch('https://origin.example.com/api/data');
    return response;
  }
}
```

Fix 4: Data Replication

```
# Replicate data to multiple regions
# Read from nearest replica

from pymongo import MongoClient

# Connect to nearest MongoDB replica
client = MongoClient([
    'us-west.mongodb.example.com',
    'eu-west.mongodb.example.com',
    'ap-south.mongodb.example.com'
], read_preference='nearest')

# Reads go to geographically closest replica
users = client.db.users.find()
```

Results:

- Without CDN: 150ms (cross-continent)
 - With CDN: 20ms (edge node)
 - Regional deployment: 10-30ms (same region)
-

2.5 Packet Loss / Network Congestion

Problem: Packets being dropped, requiring retransmission

Identify:

```
# Check packet loss
ping -c 100 api.example.com | grep "packet loss"

# Output: 5% packet loss ← Problem!

# Check network congestion
iperf3 -c api.example.com
```

Causes:

- Network congestion
- Faulty hardware
- Firewall dropping packets
- MTU size mismatch

Fix 1: Increase Timeout and Retry

```
import requests
from requests.adapters import HTTPAdapter
from urllib3.util.retry import Retry

session = requests.Session()

# Retry on transient failures
retry = Retry(
    total=3,
    backoff_factor=0.3,
    status_forcelist=[500, 502, 503, 504]
)
adapter = HTTPAdapter(max_retries=retry)
session.mount('http://', adapter)
session.mount('https://', adapter)

# Will retry on packet loss
response = session.get('http://api.example.com/users', timeout=5)
```

Fix 2: Optimize MTU (Maximum Transmission Unit)

```
# Find optimal MTU
ping -M do -s 1472 api.example.com

# Set MTU
sudo ifconfig eth0 mtu 1500

# Or in Dockerfile
RUN echo "net.ipv4.tcp_mtu_probing=1" >> /etc/sysctl.conf
```

Fix 3: TCP BBR Congestion Control

```
# Use BBR (Better congestion control algorithm)
sudo sysctl -w net.core.default_qdisc=fq
sudo sysctl -w net.ipv4.tcp_congestion_control=bbr
```

Fix 4: Quality of Service (QoS)

```
# Prioritize application traffic
# Configure on router/switch

# Mark packets with higher priority
iptables -A OUTPUT -p tcp --dport 443 -j DSCP --set-dscp 46
```

Results:

- Packet loss: 5% → 0%
 - Latency: 200ms (with retrans) → 80ms (no retrans)
-

2.6 Small File / Request Overhead

Problem: Many small requests instead of few large ones

Identify:

```
import time

# BAD: 100 small requests
start = time.time()
for i in range(100):
    response = requests.get(f'http://api.example.com/user/{i}')
total = time.time() - start
print(f"100 requests: {total:.1f}s") # 5.0s
```

Fix 1: Batch Requests

```
# GOOD: 1 large request
start = time.time()
user_ids = list(range(100))
response = requests.post(
    'http://api.example.com/users/batch',
    json={'ids': user_ids}
)
total = time.time() - start
print(f"1 batch request: {total:.1f}s") # 0.5s (10x faster)
```

Fix 2: HTTP/2 Multiplexing

```
import httpx
import asyncio

# HTTP/2: Parallel requests over single connection
async def fetch_all():
```

```
async with httpx.AsyncClient(http2=True) as client:
    tasks = [
        client.get(f'http://api.example.com/user/{i}')
        for i in range(100)
    ]
    responses = await asyncio.gather(*tasks)
    return responses

# All 100 requests over 1 TCP connection
responses = asyncio.run(fetch_all())
```

Fix 3: GraphQL (Single Endpoint)

```
# Instead of multiple REST endpoints
# GET /users/1
# GET /users/1/orders
# GET /users/1/profile

# Single GraphQL query
query {
  user(id: 1) {
    name
    email
    orders {
      id
      total
    }
    profile {
      bio
    }
  }
}
```

Results:

- 100 REST requests: 5000ms
- 1 batch request: 500ms (10x faster)
- HTTP/2 (parallel): 800ms (6x faster)

2.7 Slow Server Response

Problem: Server taking long to process request

Identify:

```
# TTFB (Time To First Byte) is high
curl -w "TTFB: %{time_starttransfer}s\n" -o /dev/null -s
http://api.example.com/users
```

```
# TTFB: 2.5s ← Server is slow!
```

This is NOT network latency - it's server processing time

Fixes:

1. Add caching layer (Redis)
2. Optimize database queries
3. Add database indexes
4. Scale server horizontally
5. Profile and optimize code

See *API-GATEWAY-PERFORMANCE-DEBUG.md* for server optimization

Step 3: Monitoring and Alerting

Real-Time Latency Monitoring

Prometheus + Grafana:

```
from prometheus_client import Histogram, start_http_server
import time
import requests

# Define metric
REQUEST_LATENCY = Histogram(
    'http_request_duration_seconds',
    'HTTP request latency',
    ['method', 'endpoint', 'status']
)

def make_request(url):
    start = time.time()
    response = requests.get(url)
    duration = time.time() - start

    # Record metric
    REQUEST_LATENCY.labels(
        method='GET',
        endpoint=url,
        status=response.status_code
    ).observe(duration)

    return response

# Start metrics server
start_http_server(8000)

# Make requests
```

```
while True:
    make_request('http://api.example.com/users')
    time.sleep(1)
```

Grafana Dashboard:

```
Queries:
- Avg latency: avg(http_request_duration_seconds)
- P50: histogram_quantile(0.5, http_request_duration_seconds)
- P95: histogram_quantile(0.95, http_request_duration_seconds)
- P99: histogram_quantile(0.99, http_request_duration_seconds)
```

Alerting

Prometheus Alert Rules:

```
groups:
- name: latency_alerts
  rules:
  - alert: HighLatency
    expr: histogram_quantile(0.95, http_request_duration_seconds) > 1
    for: 5m
    labels:
      severity: warning
    annotations:
      summary: "High latency detected (P95 > 1s)"

  - alert: VeryHighLatency
    expr: histogram_quantile(0.95, http_request_duration_seconds) > 3
    for: 2m
    labels:
      severity: critical
    annotations:
      summary: "Very high latency (P95 > 3s)"
```

Step 4: Load Testing

Test latency under different loads:

```
# locustfile.py
from locust import HttpUser, task, between
import time

class LatencyTest(HttpUser):
    wait_time = between(1, 3)
```



```
def on_start(self):
    self.request_times = []

    @task
    def get_users(self):
        start = time.time()
        response = self.client.get("/users")
        duration = time.time() - start

        # Track latency
        print(f"Latency: {duration*1000:.0f}ms")
```

Run:

```
# Start with 10 users, increase to 100
locust -f locustfile.py --host=http://api.example.com

# Monitor how latency changes with load:
# 10 users: 50ms avg
# 50 users: 100ms avg
# 100 users: 500ms avg ← Degradation
```

Optimization Checklist

Quick Wins (Low Hanging Fruit)

- ☐ Enable connection pooling / keep-alive
- ☐ Use HTTP/2
- ☐ Enable compression (gzip)
- ☐ Batch requests where possible
- ☐ Add DNS caching

Medium Effort

- ☐ Implement CDN for static assets
- ☐ Add Redis caching layer
- ☐ Enable TLS session resumption
- ☐ Optimize database queries
- ☐ Add database indexes

High Effort

- ☐ Deploy to multiple regions
- ☐ Implement edge computing
- ☐ Database replication (read replicas)
- ☐ Migrate to HTTP/3 (QUIC)
- ☐ Network optimization (BBR, QoS)

Latency Budget Example

Target: 500ms total latency

Component	Budget	Optimized
DNS lookup	50ms	5ms (caching)
TCP connect	50ms	0ms (keep-alive)
TLS handshake	80ms	10ms (session resumption)
Request transmission	20ms	20ms
Server processing	200ms	100ms (optimization)
Response transmission	50ms	30ms (compression)
Network buffer	50ms	50ms
TOTAL	500ms	215ms ✓

Interview Answer Template

"How would you debug and fix network latency?"

Step 1: Measure

- Use curl timing to break down latency components
- Identify where time is spent (DNS, TCP, TLS, server, transfer)

Step 2: Identify root cause

- High DNS time → DNS caching
- High TCP time → Keep-alive, geographic distance
- High TLS time → Session resumption
- High TTFB → Server issue (not network)
- High transfer → Payload size, compression

Step 3: Fix

- Quick wins: Connection pooling, HTTP/2, compression
- Geographic: CDN, regional deployments
- Server: Caching, optimization, scaling

Step 4: Monitor

- Prometheus metrics
- Alert on P95 latency
- Load testing

Example: "Found TTFB was 2s. This indicated server processing, not network. Added Redis caching and optimized queries. Latency dropped from 2.5s to 200ms total."

Tools Summary

Measurement:

- `curl` - timing breakdown
- `ping` - basic latency
- `traceroute` / `mtr` - hop-by-hop analysis
- OpenTelemetry / Jaeger - distributed tracing

Monitoring:

- Prometheus - metrics collection
- Grafana - visualization
- Datadog / New Relic - APM

Load Testing:

- Locust - Python load testing
- k6 - Modern load testing
- Apache JMeter - Traditional

Network:

- `netstat` / `ss` - connection state
 - `tcpdump` / Wireshark - packet analysis
 - `iperf` - bandwidth testing
-

Real-World Example

Problem: API response time is 3 seconds

Investigation:

```
curl -w "@curl-format.txt" https://api.example.com/users

time_namelookup: 0.150s ← High DNS!
  time_connect: 0.200s
time_appconnect: 0.280s
time_starttransfer: 2.500s ← High TTFB!
  time_total: 3.000s
```

Root Causes:

1. DNS: 150ms (no caching)
2. Server: 2.2s (slow query)
3. No connection reuse

Fixes Applied:

1. DNS caching → 150ms → 5ms
2. Added Redis cache → 2.2s → 50ms
3. Connection pooling → saved 80ms per request
4. HTTP/2 → multiplexed requests

Results:

- Before: 3000ms
 - After: 150ms (20x faster!)
 - P95 latency: 200ms
 - Throughput: 10x increase
-

Summary

Latency = DNS + TCP + TLS + Server + Transfer

Key Optimizations:

1. **Cache DNS** (50ms → 5ms)
2. **Keep-Alive** (eliminate repeated TCP/TLS)
3. **HTTP/2** (multiplex requests)
4. **Compression** (reduce transfer time)
5. **CDN** (reduce geographic distance)
6. **Caching** (reduce server time)

Measurement is critical - can't optimize what you don't measure!